

PRÁCTICA 1

INTEGRACIÓN DE MICRO-ROS EN ESP32-S3 CON PLATFORMIO Y VISUAL STUDIO CODE

Juan Pedro Bandera Rubio (jpbandera@uma.es)

Trabajar con robots, hoy día, implica saber al menos qué es y cuáles son las bases del funcionamiento de ROS2 (Robot Operating System, versión 2). En esta asignatura ya os hemos explicado estos conceptos a nivel más teórico. También os hemos presentado a Micro-ROS, la versión de ROS2 para microcontroladores que puedes integrar en tus códigos para el ESP32-S3.

Sin embargo, encontramos aquí una primera barrera que superar: ***tanto ROS2 como Micro-ROS están fundamentalmente pensados para trabajar en el Sistema Operativo Linux.***

En los PCs del laboratorio está instalado el SO Linux [Ubuntu](#) (versión actual, 24.04). También se ha instalado ROS2 (distribución actual, [jazzy](#)). Si quieres emplear Micro-ROS en tu ESP32-S3, y conectarlo a otros microbots que usen Micro-ROS o a PCs externos ejecutando ROS2, tendrás que instalar en tu PC o portátil una partición con Linux y ROS2 (se recomienda instalar la versión de Ubuntu y la distribución de ROS2 indicadas).

Lo bueno es que Visual Studio Code y PlatformIO funcionan exactamente igual en Linux que en Windows, así que una vez los instales trabajarás igual en ambos sistemas operativos.

INSTALACIÓN DE LINUX (UBUNTU)

En principio puedes usar el Linux que quieras, no tiene porqué ser Ubuntu, ni tampoco tiene porqué ser Ubuntu 24.04. Sin embargo, Ubuntu es la distribución más popular de Linux actualmente, y su versión de escritorio (Ubuntu Desktop) una de las más fáciles de usar.

A pesar de lo anterior, la instalación de Ubuntu hay que hacerla con cuidado, para no fastidiar tu sistema operativo actual. Las instrucciones están en este enlace: <https://ubuntu.com/desktop/docs/en/latest/tutorial/install-ubuntu-desktop/> y es verdad que, como dicen, la cosa no es complicada. Pero sí es recomendable, insistimos,

ir con cuidado. Sobre todo ten en cuenta, si ya tienes otro sistema operativo en tu disco (como por ejemplo, Windows), que cuando llegue el momento de instalar Ubuntu tendrás que escoger ***Installing Ubuntu alongside another operating system*** (o su equivalente en castellano), y asignarle una cantidad de memoria adecuada a la partición donde se instalará (150-200 GB deberían ser más que suficientes). Ese espacio dejará de estar disponible para tu SO Windows, claro.

Al finalizar la instalación, se habrá incorporado un gestor de arranque a tu PC que te preguntará, cada vez que inicies o reinicies, qué sistema operativo quieres usar. Seguirás usando Windows como hasta ahora, o podrás arrancar y usar Linux.

INSTALACIÓN DE ROS2

Es posible instalar ROS2 en Windows. Sin embargo, para empezar, el proceso de instalación y uso es más complejo. Y luego, está el hecho de que, siendo ROS2 algo evidentemente pensado para Linux, cuando lo consigues echar a andar en Windows empezarás a encontrar que hay paquetes, códigos en GitHub, funcionalidades... que se han realizado sin tener en cuenta compatibilidad con Windows. Si tienes un conocimiento experto en estos temas es más que probable que puedas configurar estos códigos para adaptarlos a ROS2 sobre Windows. En otro caso tendrás que renunciar a usarlos.

El componente para PlatformIO IDE en VSC, que permite integrar Micro-ROS en tu ESP32-S3, es un ejemplo de este tipo de códigos, que es más o menos directo de usar sobre Linux, pero NO sobre Windows. Es por eso que creemos que lo mejor en la asignatura de microbótica es que instaléis ROS2 sobre Ubuntu.

Siguiendo la documentación de la distribución jazzy de ROS2, que es la que queremos usar (<https://docs.ros.org/en/jazzy/index.html>), puedes instalarla a partir del paquete .deb (opción recomendada) como se indica aquí: <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>. Si eres totalmente nuevo en Linux, los cuadraditos verdes que aparecen en esta web de instalación son lo que tienes que escribir en una terminal, o consola, de Linux. Si tienes dudas sobre esto, consúltanos a los profesores de la asignatura.

INSTALACIÓN DE VISUAL STUDIO CODE Y PLATFORMIO EN UBUNTU

La instalación de Visual Studio Code en Ubuntu es muy fácil. Tal y como se indica aquí: <https://code.visualstudio.com/docs/setup/linux>, descargas el paquete .deb

correspondiente y lo instalas con la orden `sudo apt install ./<nombre_paquete>.deb` ejecutada desde un terminal.

Una vez tengas VSC instalado, la instalación y gestión de extensiones (como PlatformIO IDE) se hace igual que en Windows. Ten en cuenta que también tendrás que copiar el archivo de configuración de placa `esp32-s3-devkitc-1-n16r8v.json` a tu correspondiente directorio `$HOME/.platformio/platforms/espressif32/boards`, tal y como se indica en la Práctica 0.

PLANTILLA PARA INTEGRAR MICRO-ROS EN TU ESP32-S3 USANDO PLATFORMIO

En AVISPA (<https://www.uma.es/avispa>), formado por investigadores del grupo TIC-125, del Dpto. de Tecnología Electrónica, centrados en la robótica y los entornos inteligentes, hemos creado en GitHub una plantilla que puedes descargar de aquí:

<https://github.com/grupo-avispa/micro-ros-esp32-template/tree/espidf>

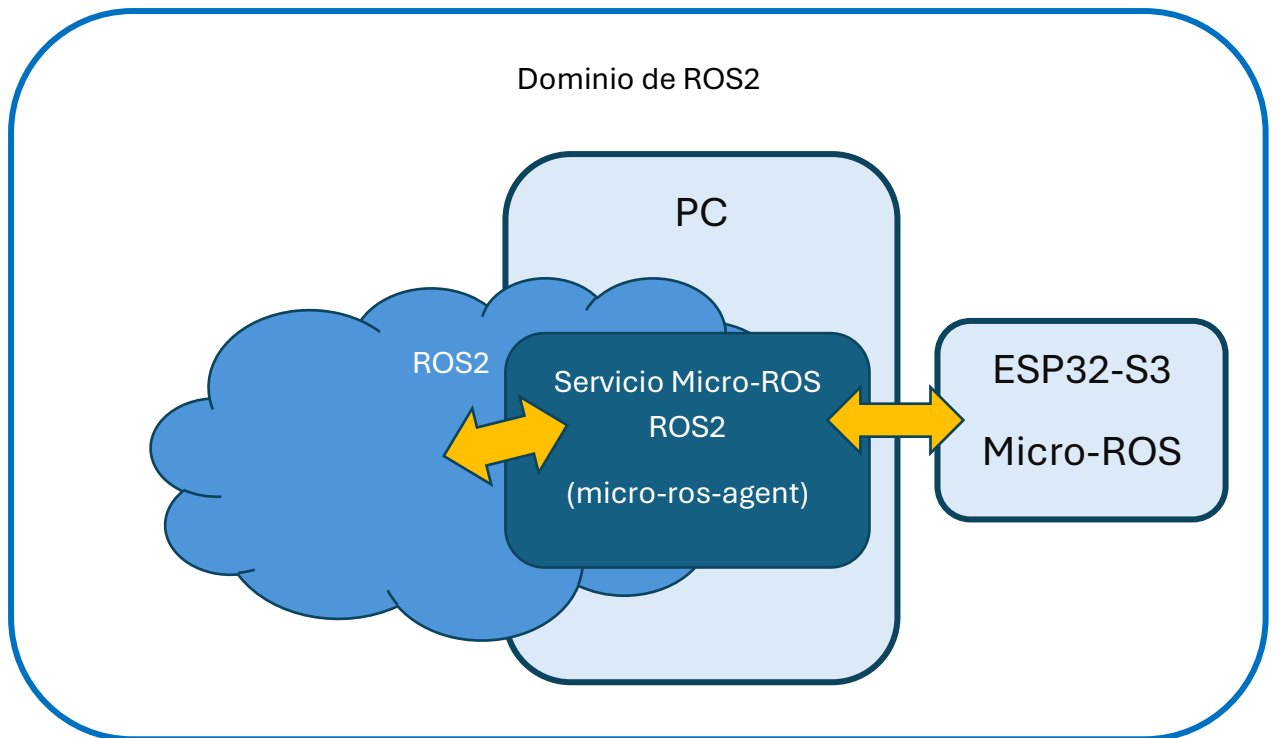
Esa plantilla es un proyecto en PlatformIO que te permitirá probar un código de ejemplo en Micro-ROS en tu ESP32-S3, usando el framework ESP-IDF.

NOTA: Si estás usando el framework Arduino, esta plantilla tiene dos versiones, una para cada framework, así que sólo tendrás que cambiar la rama de GitHub o directamente descargar la plantilla para framework Arduino de aquí:

<https://github.com/grupo-avispa/micro-ros-esp32-template/tree/arduino>

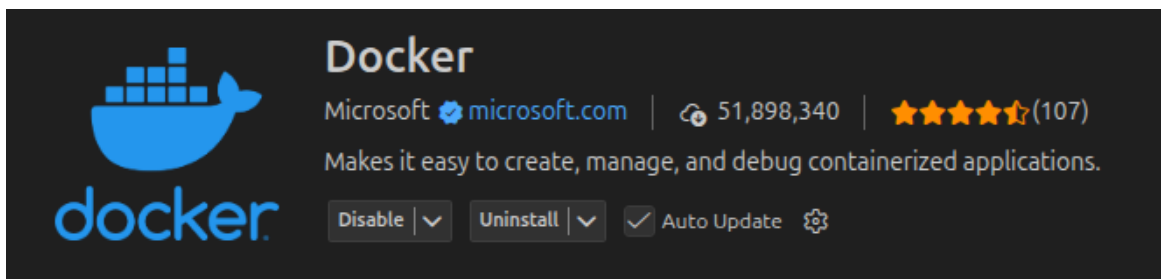
NOTA: Recuerda que la forma de 'descargar' algo de GitHub es clonarlo en tu PC.

Antes de poder ejecutar el código de esta plantilla, conviene tener en cuenta lo que va a ocurrir cuando ejecutes Micro-ROS en tu ESP32-S3. Básicamente, va a ser algo así:



Micro-ROS se comunicará con un servicio que servirá de pasarela con el resto de nodos ROS2. Este servicio, llamado **micro-ros-agent**, puedes lanzarlo desde la plantilla. Para ello, una vez tengas la plantilla en tu PC, abre el directorio (igual que harías en cualquier otro proyecto de PlatformIO) para cargar el proyecto. Luego, tienes que:

1. Instalar la extensión Docker en VSC.



2. Configurar en el archivo `include/config_transport.hpp` tus parámetros de conexión. Si vas a usar el servicio de pasarela con ROS2 desde tu PC, podría ser así:

```
/// WiFi network SSID (network name).
static constexpr char WIFI_SSID[] = "<nombre_wifi>";

/// WiFi network password (WPA/WPA2).
static constexpr char WIFI_PASSWORD[] = "<password_wifi>";

/// Maximum number of WiFi connection retries before giving up.
static constexpr int WIFI_MAXIMUM_RETRY = 10;

/// IP address of the micro-ROS Agent (dotted-decimal string).
```

```
static constexpr char AGENT_IP[] = "127.0.0.1"; //local PC

/// UDP port of the micro-ROS Agent (string, as required by the micro-ROS API).
static constexpr char AGENT_PORT[] = "9999";
```

3. Configurar en el archivo *include/config_ros.hpp* el dominio de ROS2 y el nombre de nodo ROS2 que se asignará a tu ESP32-S3:

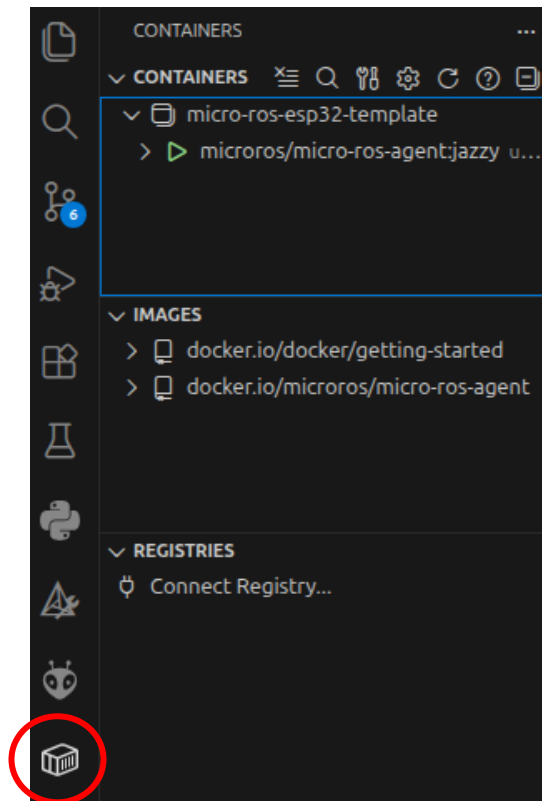
```
/// ROS 2 Domain ID for network isolation (0-232)
static constexpr uint32_t ROS_DOMAIN_ID = 11; // por ejemplo

/// ROS 2 Node name
static char ROS_NODE_NAME[] = "esp32_s3_node"; // por ejemplo
```

4. Configurar el servicio. En el archivo *docker-compose.yml* básicamente cambia los valores de *ROS_DOMAIN_ID* al número que has puesto en el paso anterior, para que el servicio esté en el dominio adecuado cuando lo arranques.
5. Arrancar el servicio. En una terminal de VSC (en el menú, *Terminal->New Terminal*, o *Ctrl+Shift+`*), escribir lo siguiente:

```
docker compose up -d udp-agent
```

6. Si todo va bien, se lanzará el servicio en un docker. Puedes comprobarlo en la pestaña 'Containers', a la izquierda de VSC. Al pulsarla, debería salir algo como lo que muestra la imagen:



NOTA: El servicio *micro-ros-agent* sólo se lanzará si ya tienes ROS2 correctamente instalado en tu sistema.

CARGA DEL CÓDIGO DE EJEMPLO DE LA PLANTILLA EN EL ESP32-S3

Tendrás que cambiar el nombre de la placa en el archivo *platformio.ini*, para que se ajuste a la placa que usamos en microbótica. Esta parte del archivo quedará así, si usas el framework *espidf*:

```
[env:esp32-s3-devkitc-1-n16r8v]
platform = espressif32
board = esp32-s3-devkitc-1-n16r8v
framework = espidf
```

(lo demás de *platformio.ini* déjalo igual).

Compila y carga el código en el ESP32-S3 como con cualquier otro código de PlatformIO. Si todo va bien, tendrás un código que se conectará a la WiFi con las credenciales que le has dado, y arrancará una tarea en el ESP32-S3 (*micro_ros_task*) para ejecutar un código que hará lo siguiente:

NOTA: El código para la conexión a la WiFi es diferente que el que te suministramos luego, en la Práctica 7. Puedes usar el que más te guste.

1. Inicializa la conexión entre el ESP32-S3 y el servicio *micro-ros-agent*.

2. Inicializar el nodo ROS2 de tu microcontrolador con el nombre que indicaste antes (en esta memoria lo hemos llamado *esp32_s3_node*).
3. Iniciar un publicador para un tópico (topic) ROS2 llamado *esp32/counter*.
4. Crear un temporizador ROS2 que interrumpe cada segundo, y arrancarlo. Aquí puedes usar cualquier otro tipo de temporizador. En las siguientes prácticas guiadas vas a recordar cómo usar temporizadores FreeRTOS, y también cómo usar el ESP32-S3.
5. El temporizador invoca al publicador cada vez que interrumpe, y se ordena la publicación del valor de cuenta en el topic *esp32/counter*.

Para comprobar que todo ha ido bien, en una terminal de tu PC, o en la terminal de VSC (recuerda, en el menú, *Terminal->New Terminal*, o *Ctrl+Shift+`*), puedes ahora escribir lo siguiente:

```
ros2 topic list
```

para tener una lista de los topics de ROS2, entre los que debería estar ya *esp32/counter*.

Y si escribes

```
ros2 topic echo esp32/counter
```

deberías ver los mensajes que ya está publicando tu ESP32-S3 en ROS2.